Patient Records Guide

This document explains, at a high level, how a patient's clinical record is stored in this codebase: the three kinds of record (findings, evaluations and procedures), what each one means, and how they fit together. It is the conceptual companion to the Clinical Rules Guide, which covers how rules query these records.

Overview

Everything clinical we know about a patient is a row in `patient_records`. A record says, in SNOMED terms, *what* it is about and (optionally) *what the answer was*:

Column	Meaning
`id`	UUID. Can be generated by the caller before insert
`patient_id`	The patient
`patient_encounter_id`	The encounter during which the record was made
`root_snomed_concept_id`	The broad kind of thing, e.g. "Clinical finding", "Diagnosis", "Procedure"
`specific_snomed_concept_id`	The particular thing, e.g. "Anaphylaxis", "Swelling", "Oxygen therapy"
`value_snomed_concept_id`	Optional answer. For findings, a Yes/No/Unknown qualifier; for diagnoses, a certainty such as "Probable"
`created_at`	When it was written

`patient_records` only holds what every record has in common. What *kind* of record it is, and the extra data that kind needs, lives in a **subtype table** whose primary key is the record's id:

Kind	Table	In one line
Finding	`patient_findings`	Something observed or reported: what the patient has, or doesn't have
Evaluation	`patient_evaluations`	Something concluded: a diagnosis, a score, a task to do, or a correction
Procedure	`patient_procedures`	Something done by a health worker, which groups the findings and evaluations made during it

A few more tables hang off these (measurements, symptoms, qualifiers, relations and so on). They are described under the kind they belong to.

```
patient ── encounter ── procedure ─┬─ finding   (what we saw)
                                   └─ evaluation (what we concluded, because of other
records)
```

Two things apply to every kind of record:

- **Records are append-only.** See the next section.
- **Records are denormalized into `patient_records_aggregated`** (one row per record, with concept names and a JSON `value`) so that rules and the UI can read them cheaply. It is derived data, filled in automatically when a record is inserted. See rules-guide.md.

SNOMED CT in Brief

Every record is described with SNOMED CT, an international clinical vocabulary. Instead of free text ("high temp", "fever", "pyrexia"), a record points at a **concept**: a numbered, unambiguous definition of one clinical idea. Concepts are what let the system, the rules and other health systems agree on what a record means.

- **A concept has an id, a name and a category.** For example `61462000` is "Malaria", category `disorder`. The category (SNOMED's *semantic tag*) says what sort of thing the concept is: `finding`, `disorder`, `procedure`, `observable entity`, `qualifier value`, `attribute`, and so on. You can look any concept up in the SNOMED International Browser, which opens on Malaria.
- **Concepts form a hierarchy.** Each concept has "is a" parents and children: "Quartan malaria" and "Mixed malaria" are kinds of "Malaria". The hierarchy is why a rule that asks about malaria also matches its more specific kinds. The database keeps the transitive closure of this hierarchy in `snomed_concept_active_descendants_realized` so that lookups are a join.
- **Concepts are also used for the *shape* of a record, not just its clinical content.** The root concept ("Clinical finding", "Diagnosis", "Procedure") and the value concept (Yes / No / Unknown, or a diagnostic certainty such as "Probable") are SNOMED concepts too. This is why `patient_records` is three concept columns plus ids.
- **Concepts used directly by our code are named in one place**, shared/snomed_concepts.ts, so code refers to `DIAGNOSIS` or `ENTERED_IN_ERROR` rather than to a bare number.

The `vha-snomed` MCP server, defined in routes/mcp/snomed.ts, lets AI agents search for concepts by name. People can use the SNOMED International Browser linked above.

An Append-Only System

The patient record is written once and never edited. Nothing in `db/models/` issues an `UPDATE` or `DELETE` against `patient_records` or its subtype tables. To change the record, you add to it:

- A new observation is a new finding. It does not overwrite the old one. "Blood pressure is now 90/60" sits next to the earlier reading.
- A mistake is corrected by writing an evaluation, "Entered in error" or "Altered", that points at the bad record. The bad record stays; it just stops counting. See Correcting the Record.
- A conclusion that no longer holds is superseded by a later evaluation, not rewritten.

This is a convention of the model code rather than something the database enforces. The tables still have an `updated_at` column (from `createStandardTable`), but no record's meaning is changed by an update.

Why it matters:

- **Clinical audit.** The record shows what was known, by whom (`employment_id`, or `by_system`), and when. Nothing has to be reconstructed from backups.
- **Rules can reason about time.** Rules ask "was this ever recorded?" or "is it still valid?" and both questions have stable answers.
- **Offline use and reconciliation (future).** A device that is offline can keep adding records and sync them later. Because rows are never edited, two copies of a patient's record can only disagree by one having rows the other lacks, and reconciling means taking the **union of the rows**, keyed by `id`. There are no field-level merge conflicts, and a re-sent row is a harmless duplicate. This works because:
  - ids are UUIDs that the caller can generate ahead of time (insert helpers take a caller-supplied `id`, e.g. when the frontend generates it), so a device doesn't need the server to name a record;
  - corrections are themselves records, so "this was a mistake" travels with the data and merges like anything else;
  - validity (`patient_records_still_valid`) and the aggregated table are derived from the records, so they can be recomputed after a merge.

  Offline mode isn't built. These are things to settle when it is:

  - `created_at` defaults to the server's `now()`, so the time a record was actually made on the device would need to be carried explicitly.
  - Only one encounter can be open per patient (`only_one_open_encounter_per_patient`), so two devices opening encounters for the same patient need a policy.
  - System-generated records (diagnoses and priorities with `by_system`, `due_to` tags) are computed when records are inserted. After a merge, rules may need to be re-run over the

combined record.

Findings

A finding is **something observed or reported about the patient**: a symptom, a sign, a history item, a vital sign, an allergy, a family-history item. It answers "what is the case with this patient?"

Table: `patient_findings`. Model: db/models/patient_findings.ts.

Every finding belongs to a procedure (`procedure_id`, required) and records which clinician on the encounter entered it (`patient_encounter_employee_id`). There are no findings that appear from nowhere: they are always found *during* something, such as triage or a vitals check.

Existence: yes, no and unknown

A finding is a statement, and the statement can be negative. The `value_snomed_concept_id` carries the answer, and the aggregated table summarizes it as `existence`:

`existence`	Meaning
`Yes`	The patient has it
`No`	The patient was asked and **does not** have it
`Unknown`	Asked, but the answer isn't known

"No" is a recorded fact, not an absence. A patient with no row for "fever" has not been asked; a patient with a "fever: No" row has. Rules rely on this difference.

Kinds of finding

Some findings carry extra structured data in their own table:

Table	Adds
`patient_measurements`	A numeric `value`, `units`, and a `comparator` (`=`, `>=`, ...) so that "≥ 90" can be stored honestly
`patient_symptoms`	`severity` (1–10), `start_date`, optional `end_date`, `notes`
`patient_chief_complaints`	The patient's own words (`note`) and its language
`patient_computed_findings`	A value derived by an algorithm, with the algorithm version and the measurements it came from
media tables	An image or a recording attached to a finding

Qualifiers

A finding can be refined by **qualifiers**, which are themselves records in `patient_record_qualifiers` that point at the finding they qualify. For example "Swelling" qualified by "Sudden onset". Qualifiers are how a rule can distinguish a sudden swelling from a chronic one without needing a separate concept for each.

Evaluations

An evaluation is **something concluded about the patient, usually because of other records**. Where a finding is data, an evaluation is judgement. It answers "what do we make of this?"

Table: `patient_evaluations`. Model: db/models/patient_evaluations.ts.

An evaluation can be made **by a person** (`employment_id` , and the `procedure_id` they were performing) or **by the system** (`by_system = true` , when a rule produced it). The database requires one or the other. An evaluation may point at the record it evaluates (`evaluates_record_id`), and further relations can be added with `patient_record_relations` .

Kinds of evaluation

- **Diagnoses.** The root concept is "Diagnosis", the specific concept is the disorder, and the value is the **certainty**:

Certainty	SNOMED concept
Definite	Definite
Probable	Probable diagnosis (contextual qualifier)
Possible	Possible diagnosis (contextual qualifier)
Equivocal	Equivocal
Improbable	Improbable diagnosis (contextual qualifier)

  The records that support a diagnosis are linked to it as **evidence**, through a `patient_record_relations` row marked "Evidence of (contextual qualifier)". Evidence links are what let the system know which diagnoses to reconsider when a finding turns out to be wrong. Certainty is part of the record, so a change of mind is a new diagnosis evaluation with a different certainty rather than an edit to the old one.

- **Scores.** An evaluation with a number (`patient_evaluation_scores`), such as a severity score.

- **Tasks.** An evaluation that says "this should be done" (`patient_record_tasks` links it to a task). This is how a rule's conclusion becomes something for the health worker to act on.

- **Corrections.** "Entered in error" and "Altered", whose job is to take another record out of play. See Correcting the Record.

Rules that *produce* evaluations (`system_diagnosis_rule`, `system_priority_evaluation`) are covered in rules-guide.md.

Procedures

A procedure is **something a health worker did**: triage, a check for a condition, a measurement session, a treatment given, a consultation, a transfer of care. It answers "what happened during this encounter?"

Table: `patient_procedures`. Model: db/models/patient_procedures.ts.

- A procedure always has an `employment_id`: the clinician who performed it.
- Procedures **contain** other records. Findings and person-made evaluations name the procedure they were made in (`procedure_id`), so "everything recorded during triage" is one query.
- Procedures can nest (`as_part_of_procedure_id`), so a step in a workflow can be a procedure inside the larger procedure it belongs to. Some procedure concepts correspond to named workflow steps in the UI.
- A procedure can carry a little extra data about what was done: a **link** (`patient_record_links`: title, URL, optional thumbnail) or an embedded **s-expression** (`patient_record_s_expressions`) describing the action.

How the Three Fit Together

The kinds form a chain: procedures produce findings, findings feed evaluations, evaluations produce tasks, and tasks lead to more procedures. Here is a hypothetical walk-through, using anaphylaxis as the condition.

1. A nurse starts a **triage procedure** for the patient's open encounter.
2. During it, the nurse records **findings**: "Swelling" (qualified "Sudden onset"), "Difficulty breathing: Yes", "Fever: No", and a blood-pressure **measurement**.
3. A rule matches these findings and the system writes an **evaluation**: "Anaphylaxis", probable, `by_system`. The findings are linked to it as evidence.
4. Another evaluation creates a **task**, such as "Check airway and give adrenaline".
5. The nurse performs a **procedure** (administering the treatment), then records new **findings** such as a fresh blood pressure.
6. Later, the nurse realizes the swelling was recorded by mistake. They mark that finding **entered in error**. The system also marks the dependent diagnosis **altered**, since its evidence has gone.

Every step added rows; no step changed one.

Correcting the Record

Because records are never edited, a correction is a new evaluation whose root concept is "Evaluation - action" and whose specific concept is one of:

Concept	Use
Entered in error	The record should never have been made
Altered	The record is superseded or no longer holds

The correction points at the target through `evaluates_record_id`. Helpers live in db/models/patient_records_base.ts: `markFindingEnteredInError`, `markEnteredInError` and `markAltered`.

Two effects follow:

- **The target stops being valid.** `patient_records_still_valid` lists the ids of records that haven't been invalidated. A row is added when any record is inserted, and a database trigger removes the target's row when an Altered or Entered-in-error evaluation is inserted for it. This is the only thing that gets deleted, and it is derived bookkeeping, not clinical data: the table can always be recomputed from the records. Queries that should only see live data join to it.
- **Dependent diagnoses are invalidated too.** If the invalidated record was evidence for a diagnosis (via "Evidence of"), the helpers also write an Altered evaluation for that diagnosis, within the same encounter. A diagnosis does not outlive the evidence it rested on.

`markFindingEnteredInError` only acts on a finding that belongs to the given patient and encounter and is still valid. If any of those fail, it does nothing and the caller returns a 400.

Quick Reference

	Finding	Evaluation	Procedure
Meaning	Observed or reported	Concluded or decided	Done
Answers	What is the case?	What do we make of it?	What was done?
Table	`patient_findings`	`patient_evaluations`	`patient_procedures`
Model	patient_findings.ts	patient_evaluations.ts	patient_procedures.ts
Made by	A clinician on the encounter	A clinician **or** the system	A clinician
Belongs to a procedure?	Always	When made by a person	Optionally nested in another
Typical value	Yes / No / Unknown	Certainty (diagnoses) or a score	None (a link or s-expression instead)
Examples	Swelling; Fever: No; BP 90/60; Penicillin allergy	Probable anaphylaxis; a task; Entered in error	Triage; Oxygen therapy; Telemedicine consultation
Rules query it with	`finding`, `clinical_finding`, `measurement`	`diagnosis`, `evaluation`	`procedure`

Supporting tables shared by all three:

Table	Role
`patient_encounters`	The visit that every record belongs to (one open per patient)
`patient_record_qualifiers`	Refines a record, e.g. "Sudden onset" on "Swelling"
`patient_record_relations`	Source → destination links between records, including "Evidence of"
`patient_records_still_valid`	Ids of records that haven't been altered or entered in error
`patient_records_aggregated`	Denormalized, derived view of every record for fast reads